Contents

1 Introduction **1**
- 1.1 Motivation . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Objectives . . . 2
- 1.4 Scope . . . 3

2 Literature Review **4**
- 2.1 Comparative Analysis of Recent Studies . . . 6
- 2.2 Research Gap . . . 7

3 Project Design **8**
- 3.1 Existing System . . . 8
- 3.2 Proposed System . . . 9
- 3.3 Critical Components of System Architecture . . . 11
  - 3.3.1 Sandbox and Container Manager . . . 11
  - 3.3.2 Run Monitoring and Logging . . . 12
  - 3.3.3 Diff Capture on a Dedicated Branch . . . 12
  - 3.3.4 Review and Human Approval Service . . . 13
  - 3.3.5 Backend API and Dashboard . . . 14
  - 3.3.6 Supporting Module: Protect . . . 14
- 3.4 System Diagrams . . . 14
  - 3.4.1 UML Diagram . . . 16
  - 3.4.2 Activity Diagram . . . 17
  - 3.4.3 Use Case Diagram . . . 18
  - 3.4.4 Sequence Diagram . . . 19

4 Project Planning and Cost Estimation **20**
- 4.1 Project Development Methodology . . . 20
- 4.2 Project Schedule and Gantt Chart . . . 20
- 4.3 Team Roles and Responsibilities . . . 21
- 4.4 Software Size Estimation . . . 22
  - 4.4.1 KLOC-Based Estimation . . . 22
- 4.5 Effort Estimation Using COCOMO . . . 22
  - 4.5.1 Basic COCOMO Model . . . 22
  - 4.5.2 Calculation . . . 23
- 4.6 Human Resource Cost . . . 23
- 4.7 Hardware, Software and Cloud Infrastructure Cost . . . 23
- 4.8 Testing, Deployment and Maintenance Cost . . . 24
- 4.9 Total Estimated Project Cost . . . 24
- 4.10 Assumptions and Cost Justification . . . 24
- 4.11 Risk Assessment and Mitigation . . . 25

5 Project Implementation **26**
- 5.1 Code Snippets . . . 26

5.1.1 Launching the Sandbox Container . . . 26
5.1.2 One-Shot Credential for Private Clones . . . 27
5.1.3 Event Logging and Off-Limits Enforcement . . . 28
5.1.4 Egress Allow/Deny Decision . . . 29
5.1.5 Branch Commit and Structured Diff . . . 30
5.1.6 Approval State Machine . . . 31
5.2 Steps to Access the System . . . 32
5.3 System Implementation . . . 33
5.3.1 Command-Line Interface . . . 33
5.3.2 Landing Page and Projects Dashboard . . . 33
5.3.3 Run History . . . 34
5.3.4 Run Detail Views . . . 35
5.3.5 Approval Decision and Submission . . . 37
5.3.6 Implementation Summary . . . 37
5.4 Timeline Sem VII . . . 38

6 Testing **40**
6.1 Software Testing . . . 40
6.2 Types of Testing . . . 40
6.2.1 Functional Testing . . . 40
6.2.2 System Testing . . . 41
6.3 Testing Results and Findings . . . 41
6.3.1 Test Environment and Configuration . . . 42
6.3.2 Analysis of the Two Failed Tests . . . 42
6.4 Sample Test Cases . . . 43
6.5 Conclusion on Testing . . . 44

7 Result and Discussions **45**
7.1 Introduction . . . 45
7.1.1 Benchmark Configuration . . . 45
7.2 Evaluation Metrics . . . 46
7.3 Performance Analysis . . . 46
7.4 Overhead with and without VaultShield . . . 48
7.5 Safety Events Captured . . . 48
7.6 Secret Exposure With and Without Protection . . . 49
7.7 Case Study: Analysis of Real Runs . . . 49
7.7.1 Execution Evidence from a Live Run . . . 49
7.8 Overall Discussion and Limitations . . . 53
7.8.1 Comparison with Existing Approaches . . . 53
7.9 Chapter Summary . . . 54

8 Conclusion **55**

9 Future Scope **56**

Bibliography **57**

List of Figures

3.1 Existing workflow – direct agent execution on the developer's host 8
3.2 Proposed VaultShield pipeline; the dashed Orchestrator stage is planned for Semester VIII and not implemented 10
3.3 Per-run egress network topology 12
3.4 Run status state machine 13
3.5 Overall system architecture with trust zones and attack paths 15
3.6 UML class diagram of the core backend entities and CLI classes 16
3.7 Activity diagram of a VaultShield run, from agent kickoff to PR creation 17
3.8 Use case diagram 18
3.9 Sequence diagram of one complete approved run 19

5.1 Sandbox container creation and teardown 27
5.2 Authenticated clone with a one-shot credential helper 28
5.3 Event records, off-limits matching and the JSON Lines writer 29
5.4 Egress decision for one request target 30
5.5 Committing the run branch and computing the diff 31
5.6 Transactional approve/reject decision 32
5.7 Terminal output of a successful vaultshield run with the mock agent . . . 33
5.8 Landing page 34
5.9 Projects dashboard 34
5.10 Run history with status filters 35
5.11 Run detail – Diff tab 35
5.12 Run detail – Timeline tab 36
5.13 Run detail – Screenshots tab 36
5.14 Run detail – Security tab 37
5.15 Run detail after approval, showing the submit command 37
5.16 Gantt chart of the VaultShield major project 39

6.1 Re-run of the three automated suites on 1 November 2026 (excerpt of summary lines) 42
6.2 The two failing tests re-run individually, with their durations (excerpt) . 43

7.1 Mean stage latency in the default configuration 47
7.2 Mean end-to-end time by configuration 47
7.3 A real vaultshield run with the egress allowlist and the mock agent . . . 50
7.4 Run artifacts: dummy credential file, event log and structured diff 51
7.5 Live dashboard: Timeline tab of the real run 51
7.6 Live dashboard: Diff tab of the real run, awaiting review 52
7.7 Approval gate refusing an unapproved run, and no leftover containers or host changes 52
7.8 Isolation probe run inside a real sandbox (excerpt of probe headings and errors) 52
7.9 Docker-backed egress, token-hygiene and tamper-resistance tests (excerpt of result lines) 52

List of Tables

2.1 Comparative analysis of the reviewed literature 6
2.2 Comparison with existing AI coding-agent security solutions 7

3.1 Comparison of the existing system and the proposed system 10

4.1 Planned project schedule for VaultShield 21
4.2 Team roles and responsibilities 22
4.3 Measured software size by component 22
4.4 Basic COCOMO estimate 23
4.5 Actual human-resource cash expenditure 23
4.6 Actual hardware, software and infrastructure cash expenditure 24
4.7 Actual testing, deployment and maintenance cash expenditure 24
4.8 Total actual project cash expenditure 24
4.9 Risk assessment and mitigation 25

5.1 Implemented in Semester VII 38
5.2 Planned for Semester VIII (not implemented) 38

6.1 Types of testing performed 40
6.2 Automated test suite results 41
6.3 Issues found and their resolution 41
6.4 Test environment and configuration 42
6.5 Sample test cases 44

7.1 Benchmark configurations (all with the mock agent) 45
7.2 Evaluation metrics 46
7.3 Stage latency in the default configuration 46
7.4 End-to-end latency by configuration 47
7.5 Running an agent with and without VaultShield 48
7.6 Safety events and evidence captured 48
7.7 Secret exposure under four protection configurations 49
7.8 Residual risks and limitations 53
7.9 Outcome of the same threats under three approaches 53

8.1 Achievement of objectives 55

2.2 Research Gap

The literature shows that coding agents can now make repository-scale changes [2], [3], [4], that their code and actions are risky in measurable ways [5], [6], [7], [8], that container security depends on configuration [11], [12], [13], and that governance research recommends human approval, legible logs and monitoring [9], [10]. Agent vendors have since shipped security features of their own: OS-level sandboxes for the commands an agent runs in Claude Code [16] and the OpenAI Codex CLI [18], a firewalled hosted environment for the GitHub Copilot cloud agent [19], local microVM sandboxes in Docker Sandboxes [20], and a Docker runtime with an event stream in OpenHands [21]. Table 2.2 compares them with VaultShield, based on their official documentation.

Table 2.2: Comparison with existing AI coding-agent security solutions

Control	Claude Code [16]	Codex CLI [18]	Copilot cloud agent [19]	Docker Sandboxes [20]	OpenHands [21]	VaultShield
Agent compatibility	Claude Code only	Codex only	Copilot only	Several agents	Its own agents	Any agent with an adapter (four built)
Isolation approach	OS sandbox for shell commands	OS sandbox	Hosted ephemeral environment	Local microVM	Docker container	Docker container per run (shared kernel)
Real .env kept from agent	–	–	Only copilot-environment secrets	–	–	Shape-preserving dummy values
Host checkout protected	–	–	Yes (hosted)	–	Configurable	Fresh clone; host never mounted
Network restricted by default	Configurable	Yes	Yes (firewall)	Configurable	–	Opt-in allowlist proxy
Monitoring and audit evidence	–	–	Session logs	–	Event stream	Event log, egress log, screenshots
Change record outside agent's control	–	–	–	–	–	Separate capture helper
Human approval before repository submission	Per-action prompts	Per-action approval policy	Review after the branch is pushed	–	Per-action confirmation	Whole change held until approved

"–" means the cited documentation does not describe the control. The comparison exposes four gaps. First, containment is tied to a vendor: each sandbox protects only its own agent, so a team using several agents has no single policy or evidence format. Second, the project's own .env sits inside the workspace that local sandboxes allow, and deny rules proved brittle in our spike (Section 7.6). Third, any activity record is produced beside the agent, so a hostile agent that tampers with Git state could misreport its change. Fourth, approval is either per action or comes after the change has already been pushed to the repository host. Existing products are ahead of VaultShield on isolation strength and default-deny networking. VaultShield's contribution is to close the four gaps together for any agent on the developer's own machine: dummy credentials and a one-shot clone token keep real secrets away, a separate trusted capture helper builds the change record, and an approval state machine holds the whole change before anything is pushed.

Chapter 3

Project Design

3.1 Existing System

Developers currently run AI coding agents in one of three ways. The most common is raw local execution, where the agent's CLI or editor extension runs in the developer's own checkout with the developer's environment, permissions and network access, and its changes appear as uncommitted edits. Some products add permission prompts or per-tool deny rules, but these are product-specific and often relaxed by users who find repeated prompts tedious. The second practice is ad-hoc containerisation, which usually bind-mounts the real working tree, so every write still lands on the host and the real .env file travels into the container. The third, development containers and cloud workspaces, is designed for convenience rather than for constraining an untrusted actor and holds nothing for review. Figure 3.1 summarises this workflow.

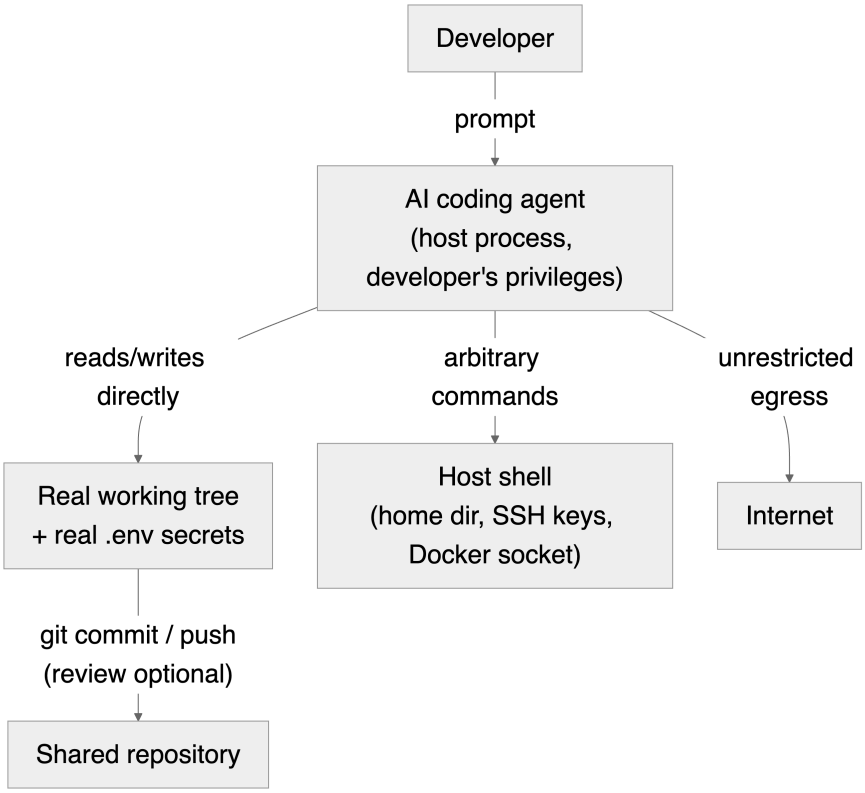


Figure 3.1: Existing workflow – direct agent execution on the developer's host

As Figure 3.1 shows, execution and acceptance are a single step and every arrow out of the agent touches a real asset. There is no isolation of side effects, no separation between the agent and the developer's secrets, no structured record of what happened, no enforced review before changes are committed, no automatic cleanup of processes the agent started,

and no record of which hosts the agent contacted.

3.2 Proposed System

VaultShield replaces this single step with a pipeline in which execution, evidence collection, storage and review are separate stages. The developer runs vaultshield run with an agent, a prompt and a repository URL. The CLI loads the project configuration, generates a run identifier, creates a per-run directory and registers the run with the backend before any container exists, so that later failures can be recorded. It then writes a dummy credential file that mirrors the shape of the developer's .env and resolves the agent adapter.

The CLI then creates a Docker container, mounts only the dummy credential file, clones the repository inside the container and installs dependencies. If egress control is enabled, the sandbox is moved onto an internal network whose only peer is a filtering proxy. Before any agent code runs, the CLI records the pre-agent commit through the Docker API as a trusted base. The agent runs under a pseudo-terminal while an in-container watcher records file changes, and both streams are merged into a timestamped event log; configured routes are screenshotted with a headless browser. The sandbox is then killed, and a separate hardened capture helper rebuilds the repository from the trusted base and a filtered copy of the work tree, commits the agent's changes to a dedicated run branch and computes the structured diff. Outputs are scanned for the agent's own credentials, the sandbox is torn down and the artifacts are uploaded, moving the run to awaiting_review. A reviewer inspects the evidence in the dashboard and approves or rejects the run, and only an approved run can be submitted as a pull or merge request. Figure 3.2 presents this pipeline.

Proposed Agentic AI Orchestrator (Objective 2). The dashed upper stage of Figure 3.2 is a proposed design for Semester VIII; it has not been implemented in Semester VII and is not part of the evaluated system. Its purpose is to replace a vague one-line prompt, which forces the agent to guess the framework, the files to change and the constraints, with a structured prompt, so that fewer changes are rejected and fewer files, APIs or packages are hallucinated. Given a task through vaultshield run --ask, a context scanner on the host, before any sandbox exists, collects a bounded summary of non-secret metadata: the directory listing to three levels, the start command and port detected by vaultshield init, dependency names, the start of the README and recent commit subjects. A sanitiser replaces protected values and secret-shaped strings with *** and blocks the request if anything secret-like remains; .env and key files are never opened. A language model then writes three to five clarifying questions in strict JSON, with one retry and a fixed fallback, and the developer answers them at the terminal or from a file. A prompt builder assembles the task, project summary, answers, a fixed Constraints section that the model cannot change, and a suggested approach. After a final secret check, the prompt is saved as prompt.md in the run directory, enters the unchanged sandboxed pipeline as the agent prompt, and is shown to the reviewer beside the diff and timeline; every model call is logged as metadata only. Because the Orchestrator sends project context to an external model and could be steered by hostile README text, it gains no privileges, its prompt is reviewed rather than trusted, and pattern-based sanitisation is a stated residual risk. Its effect on task success, approval rate and hallucinated content will be measured on real agents in Semester VIII.

Table 3.1 contrasts the proposed system with the existing practices.

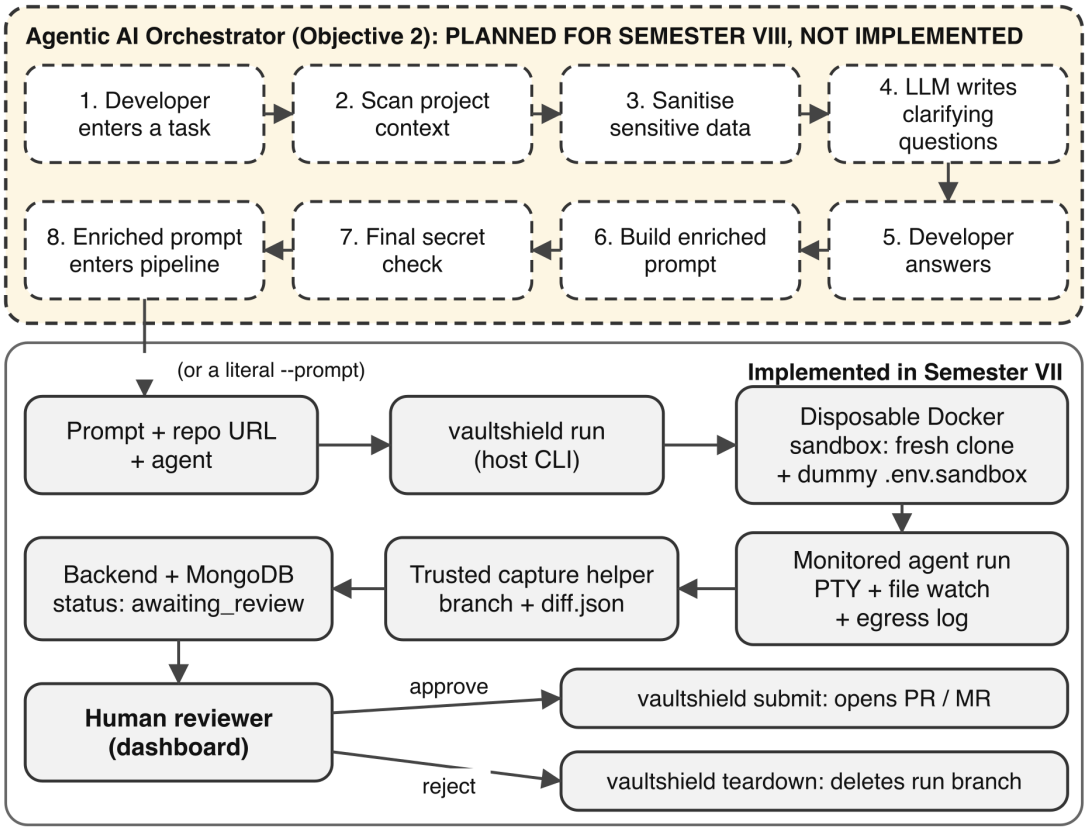


Figure 3.2: Proposed VaultShield pipeline; the dashed Orchestrator stage is planned for Semester VIII and not implemented

Table 3.1: Comparison of the existing system and the proposed system

Concern	Existing practice	Proposed system
Working tree	Real checkout, modified in place	Fresh clone inside the container; host tree never mounted
Secrets	Real .env readable by the agent	Shape-preserving dummy values in a read-only .env.sandbox
Repository token	Often visible to the agent	Given only to the single git clone process
Network	Unrestricted, unlogged	Optional allowlist through a hardened proxy, every decision logged
Evidence	Terminal scroll-back	Event log, egress log, screenshots and structured diff
Change capture	Uncommitted edits	One commit on a run branch built by a separate helper
Review	Optional and informal	Enforced state machine; PR only from approved

Concern	Existing practice	Proposed system
Cleanup	Manual	Guaranteed teardown and a scheduled reaper

3.3 Critical Components of System Architecture

The architecture has three planes: a host-side CLI pipeline, a per-run Docker execution plane, and a backend and dashboard review plane. The critical components are described below, together with the threats each one addresses.

Threat model. The protected assets are the developer's real secrets (application .env, SSH keys, repository-host token), the working tree and host, the shared repository, and the integrity of the evidence shown to the reviewer. The adversary is the agent and anything that can influence it: an agent that is simply wrong, one steered by indirect prompt injection in the repository's README, issues or dependencies [7], or a hostile agent binary or install script; it controls every process inside the sandbox. The host operating system and Docker daemon, the CLI, the capture-helper image, the backend and the repository host are trusted, and in the single-user system the developer and reviewer are one trust domain. Figure 3.5 marks the trust boundary around the untrusted sandbox and the attack paths T1 to T10, each classified below by STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege). Every mitigation named is implemented and tested in Semester VII; Semester VIII improvements appear only as residual risk. T1, prompt injection (T, I), is not prevented directly: its effects are contained by the controls below and by human approval of every change.

3.3.1 Sandbox and Container Manager

The sandbox manager owns exactly one Docker container per run. It verifies that the credential file exists, creates a container named after the run, mounts the dummy credential file read-only, clones the repository inside the container and installs dependencies. The agent is executed through exec calls rather than as the container's main process, so the container's lifetime bounds every process the agent starts. For private repositories, the repository-host token is passed only as the environment of the single git clone process, so it never appears in the container's environment, the clone's Git configuration or any later process, and it is sent only to hosts on a strict per-provider allowlist. The Docker socket is never mounted.

Threats addressed. T2, reading the real .env, SSH keys or home directory (I), fails because only the dummy file is mounted (TC-01, TC-02). T7, damage to the host or lingering processes (T, D), is contained by exec-only execution and guaranteed teardown, although no resource limits are set. T8, container escape (E), is narrowed because the Docker socket is never mounted and the capture helper drops all capabilities; the shared kernel and root user in the default sandbox remain residual risks, addressed by stronger isolation in Semester VIII.

When egress control is enabled, the EgressGuard builds the per-run topology shown in Figure 3.3: an internal network with no gateway, an ordinary bridge, and a hardened proxy attached to both. The sandbox is moved onto the internal network before the agent runs, so its only reachable peer is the proxy, which checks every destination host against

the allowlist and logs each decision. This blocks T3, exfiltration over HTTP, raw TCP or DNS (I), in 4 of 4 tested cases (TC-03); egress is off unless enabled, and uploads to an allowlisted host remain possible.

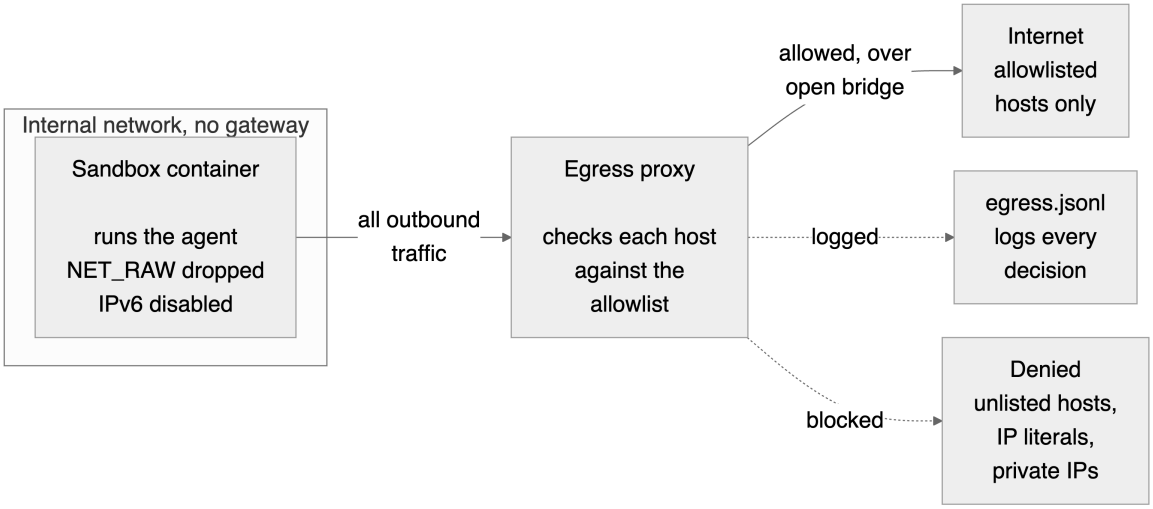


Figure 3.3: Per-run egress network topology

Any error during creation removes the partial container and raises a single SandboxError whose message never contains a token. Teardown force-removes the container and is called from a finally block, so it runs on success, failure, timeout and interruption alike.

3.3.2 Run Monitoring and Logging

The monitor produces the run's event log, events.jsonl, in which each line records an event type, a timestamp and a payload. The three event types are terminal output from the agent's pseudo-terminal, file changes observed by an in-container watcher, and violations such as off-limits writes, egress denials and leaked credentials. Each changed path is checked against the project's off-limits paths, and a match fails the run only after the log has been written, so evidence always outlives the failure. Browser capture starts the application inside the sandbox and takes one screenshot per configured route. Token hygiene scans the diff, the event log and changed files for the agent's own credential bytes; a hit fails the run, quarantines the diff and scrubs uploaded payloads.

Threats addressed. T4, leakage of the repository token or the agent's model key into outputs (I), is detected for all five tested vectors (TC-06); deliberately encoded leaks remain residual. T6, edits to protected paths such as CI files (T), is recorded and fails the run (TC-04), but only for listed paths. The event log answers repudiation (R) by recording what the agent did; the polling watcher's missed writes are addressed by improved monitoring in Semester VIII.

3.3.3 Diff Capture on a Dedicated Branch

If the branch and diff were built inside the agent's own sandbox, a hostile agent could tamper with Git state to misrepresent its changes. VaultShield therefore reads the pre-agent commit through the Docker API before the agent runs, and after the agent finishes it kills the sandbox and extracts a filtered archive of the work tree. A fresh capture helper

container with no network, no capabilities and no environment rebuilds the repository from the trusted base, refuses unsafe Git configuration, commits every change to the run branch as exactly one commit, and writes diff.json with per-file additions, deletions and size-capped patches. The branch is exported as a Git bundle because the helper is destroyed at the end of the run. This defeats T5, tampering with Git state, hooks or configuration to hide a change (T, R), in both tested variants (TC-05).

3.3.4 Review and Human Approval Service

The Run.status field is a state machine, shown in Figure 3.4. The CLI may move a run from running to awaiting_review or failed, and the approve and reject endpoints change the status only if it is still awaiting_review, inside a MongoDB transaction that also creates the Approval record. Of two concurrent decisions exactly one succeeds, and the other receives HTTP 409 with the run's actual status.

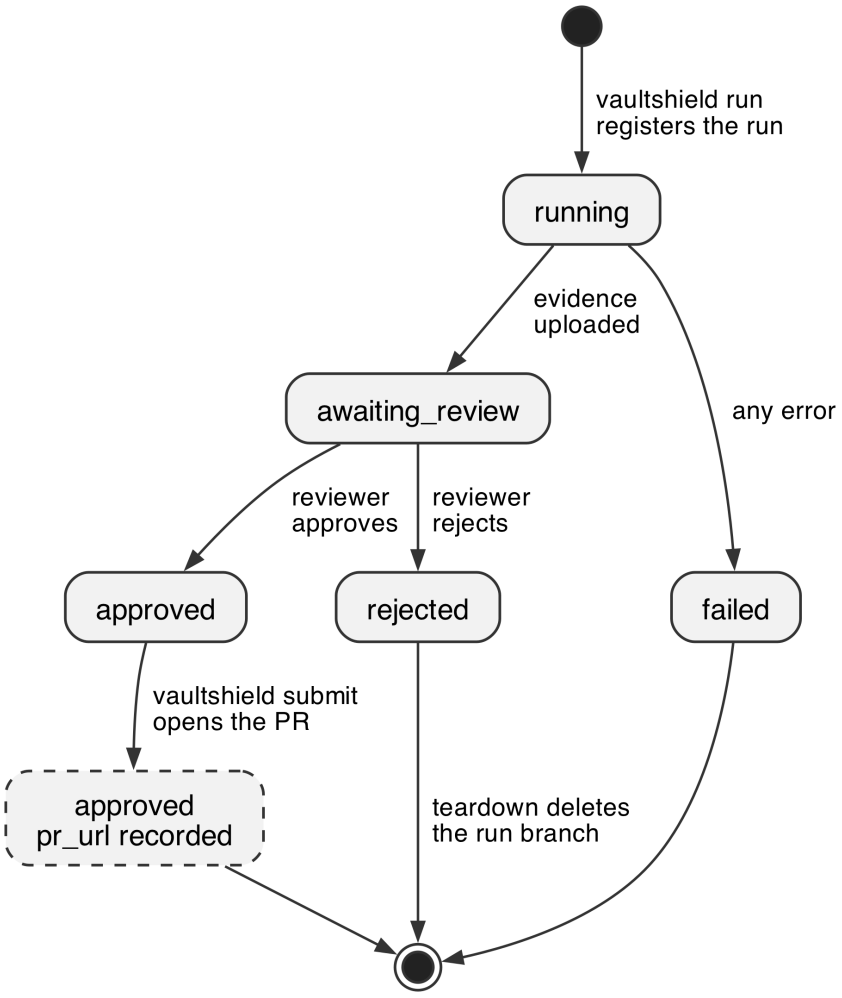


Figure 3.4: Run status state machine

vaultshield submit re-reads the run and refuses before any side effect unless its status is approved. For an approved run it checks that the current checkout matches the original repository, imports the branch from the verified bundle, pushes it without force, opens the pull or merge request and records its URL on the run. vaultshield teardown deletes the branch of a rejected run. Together these prevent T9, an unapproved change reaching the

repository or two reviewers racing (E, R) (TC-07, TC-08). Because the status PATCH route can still set approved and there is one shared identity, role-based approval is planned for Semester VIII.

3.3.5 Backend API and Dashboard

The backend is a Django 5.2 project with Django REST Framework and MongoDB as its system of record. Every API path except login requires token authentication. The CLI registers runs, uploads events, diffs and screenshots, and patches the status; the dashboard reads run details and records decisions. Token authentication, the run identifier that binds sandbox, branch and record, and the bundle's SHA-256 limit T10, altered or misattributed evidence (T, S), although any holder of the API token can upload. Celery workers with Redis run a static security scan over the added lines of each diff, reap orphaned sandboxes every 15 minutes and flag approved but unsubmitted runs. The dashboard is a React 19 application built with Vite 7 that shows a projects overview, a filterable run history and a run detail page with Diff, Timeline, Screenshots and Security tabs and the decision controls.

3.3.6 Supporting Module: Protect

Protect covers agents that developers still run outside the sandbox: it moves every .env value into the macOS Keychain, leaves a placeholder in the file and injects the real values only into the application started through vaultshield exec.

Figure 3.5 shows all components and the data flows between them. The CLI is the only component that talks to the Docker daemon, the dummy credential file is the only host path mounted into the sandbox, artifacts reach the backend over authenticated HTTP, and the submit command is the only component that pushes to the repository host, after confirming approval with the backend.

3.4 System Diagrams

System diagrams give a visual representation of the structure, behaviour and interactions of the proposed VaultShield system. The diagrams used are the UML Diagram, Activity Diagram, Use Case Diagram and Sequence Diagram.

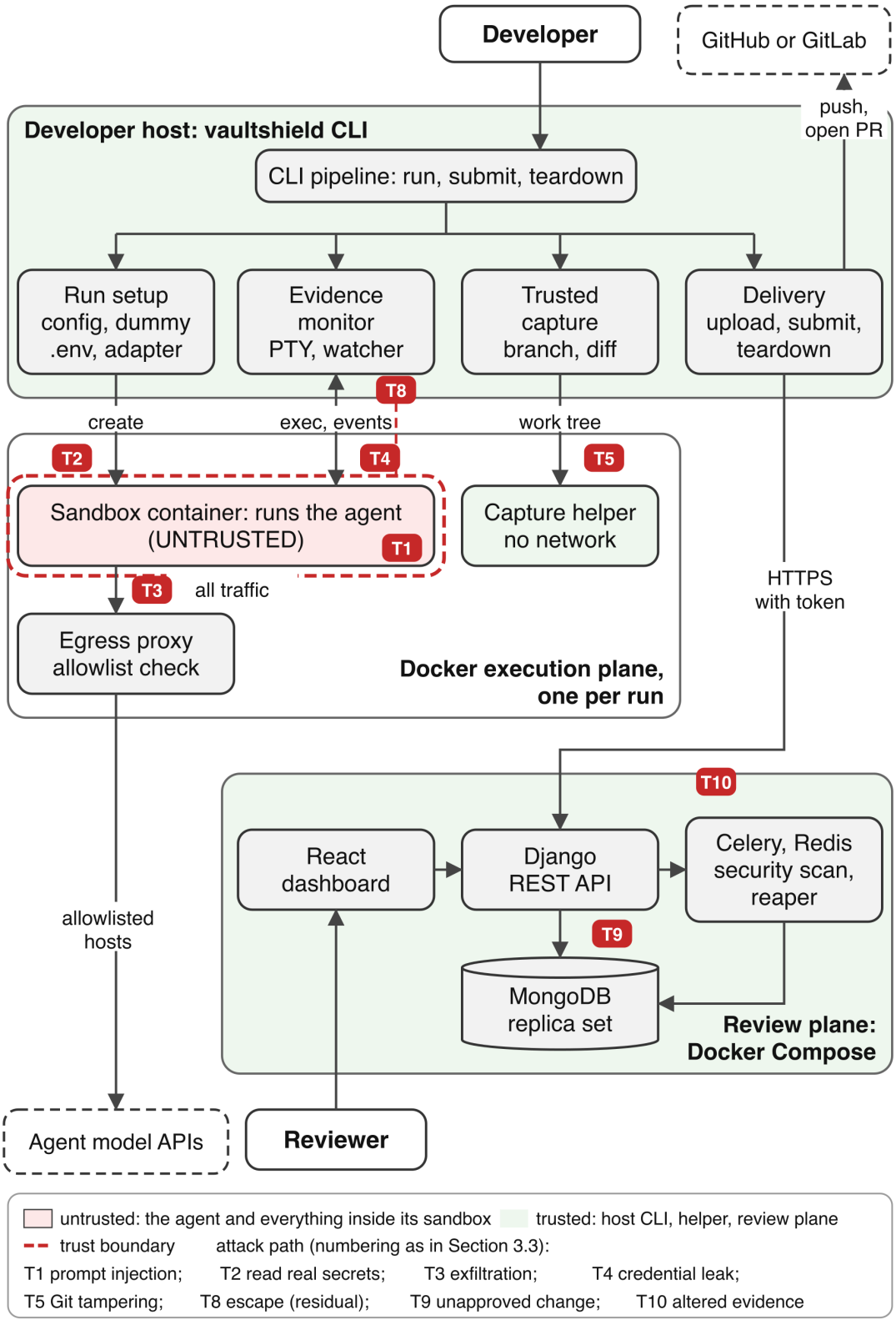


Figure 3.5: Overall system architecture with trust zones and attack paths

3.4.1 UML Diagram

Figure 3.6 shows the persistent backend entities and the principal CLI-side classes. Run is the hub of the schema: RunEvent, Screenshot and SecurityFinding are many-per-run, while RunDiff and Approval are one-to-one, which makes "a run has exactly one change set and at most one decision" a schema-level guarantee. No backend field can store a repository-host token. On the CLI side, an AgentAdapter describes how its agent runs inside the Sandbox, the EgressGuard isolates the Sandbox's network, monitoring produces the RunEvent records, and the TrustedCapture helper produces the file entries stored in RunDiff.

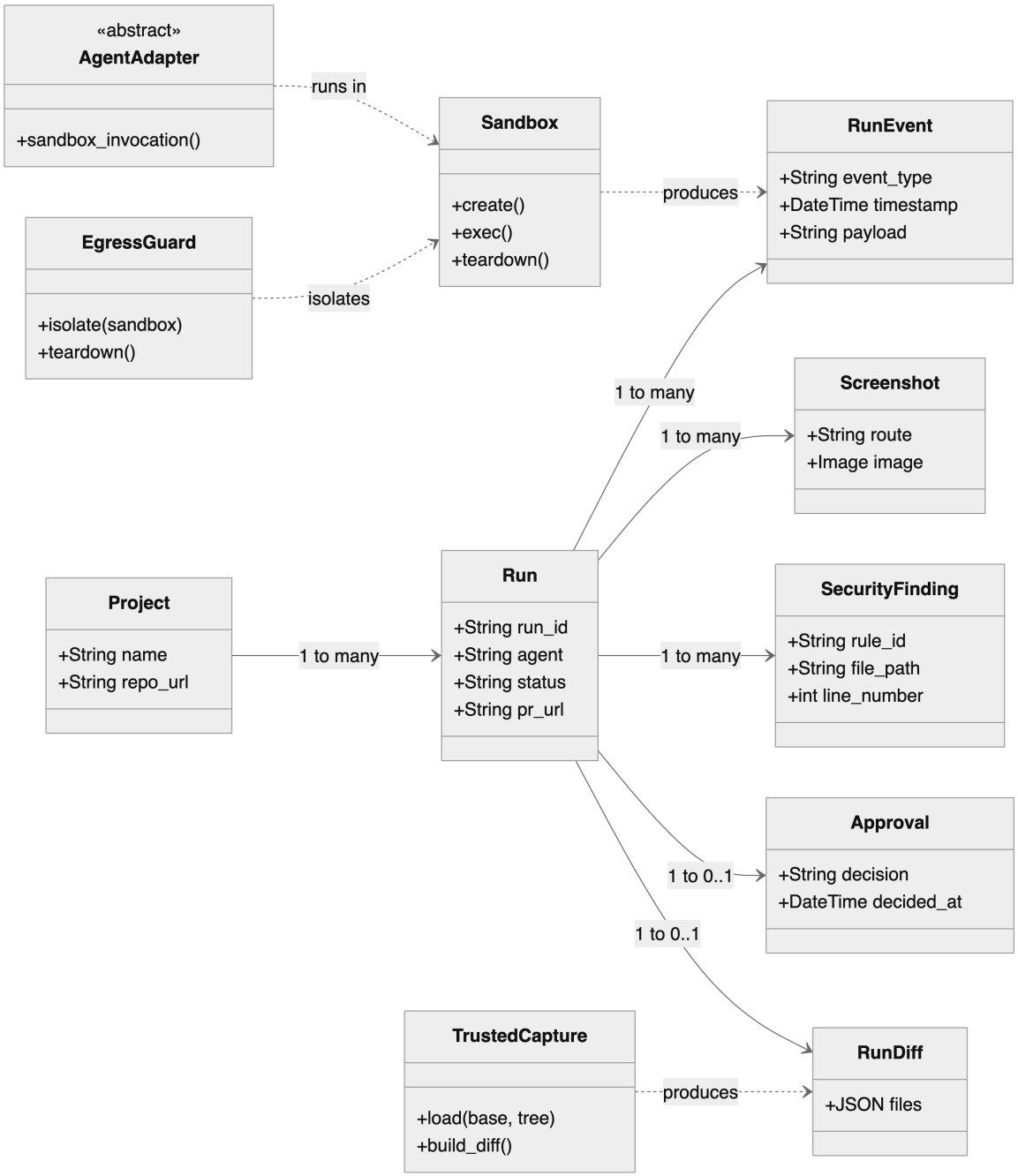


Figure 3.6: UML class diagram of the core backend entities and CLI classes

3.4.4 Sequence Diagram

Figure 3.9 shows the message exchange for one approved run. Messages 1 to 9 cover the run, 10 and 11 the review, and 12 to 16 the submission. The backend learns of the run before Docker creates anything, the branch and diff are produced by a different container from the one the agent controlled, and submit re-reads the status immediately before pushing.

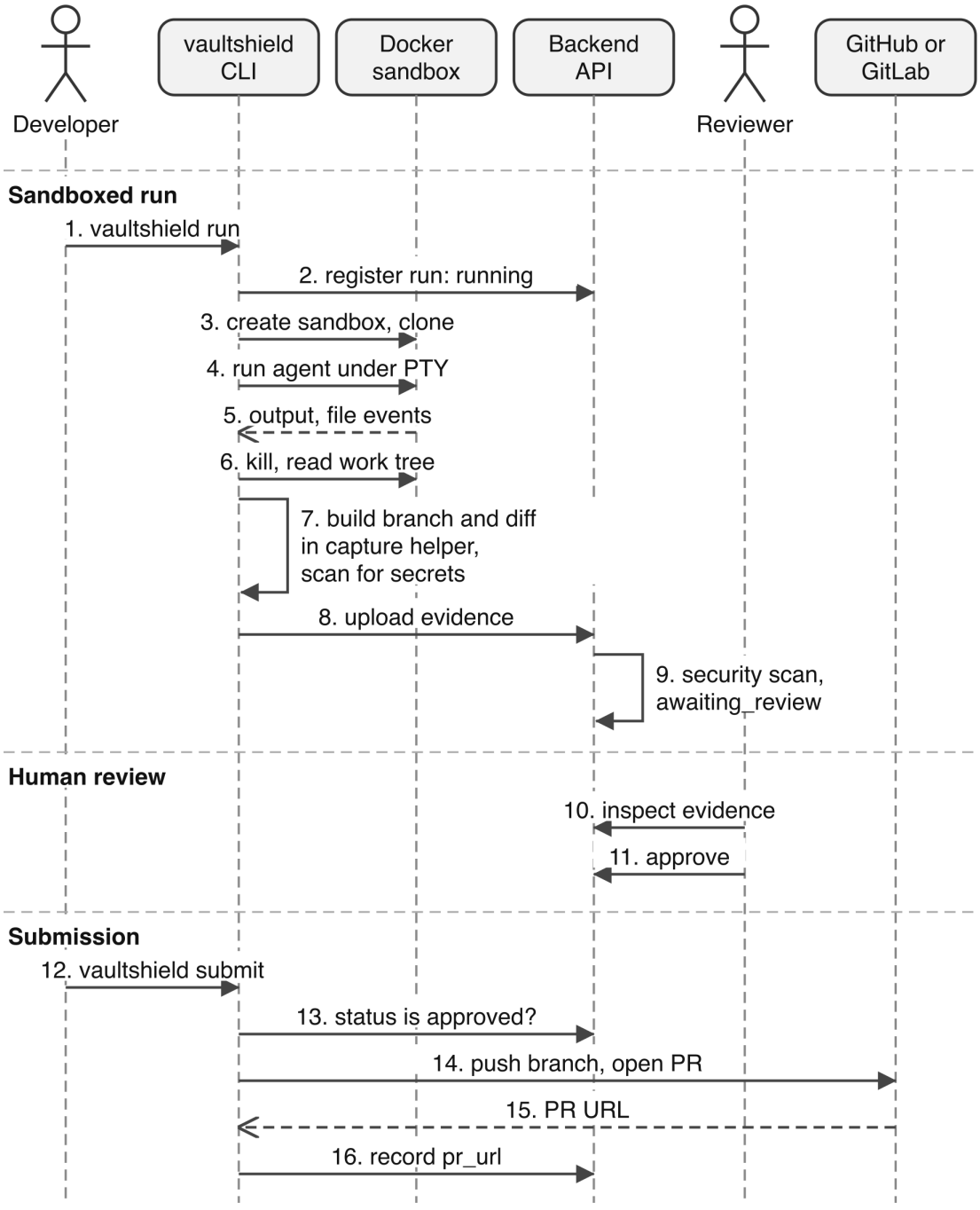


Figure 3.9: Sequence diagram of one complete approved run

Chapter 4

Project Planning and Cost Estimation

4.1 Project Development Methodology

VaultShield was developed with an iterative and incremental methodology based on specification-driven development. The team first wrote a requirements specification in which every requirement carried numbered acceptance criteria, and a design document with the architecture, data model and numbered correctness properties. The design was split into small, testable tasks, and each short iteration implemented a slice of tasks together with unit and property-based tests and, where isolation was involved, a Docker-backed integration test. The major phases were:

1. **Problem Study and Feasibility Spike:** survey the literature and measure how a production agent handles .env secrets under four protection configurations.
2. **Requirements and System Design:** write numbered requirements and define the architecture, run lifecycle, data model, REST API and threat model.
3. **CLI Core and Evidence Capture:** implement configuration, credential shaping, the sandbox lifecycle, agent adapters, monitoring, screenshots and Git capture.
4. **Backend and Dashboard:** implement the models, REST API, authentication, security scan, scheduled tasks and the review dashboard.
5. **Submission and Hardening:** implement PR and MR submission and teardown, then add the egress proxy, the trusted capture helper and token hygiene.
6. **Migration and Protect Module:** move the backend to MongoDB with transactional approval, and add the Protect module.
7. **Evaluation and Documentation:** run the tests and performance benchmark and write this report.

4.2 Project Schedule and Gantt Chart

The Semester VII activities follow the project Gantt chart, reproduced as Figure 5.16, and the Semester VIII activities are planned extensions. Table 4.1 lists each activity with its duration and target period.

In Semester VIII the project continues with four planned enhancements, none of which is part of the Semester VII system. The first is the Agentic AI Orchestrator of Objective 2, which will scan the project, ask the developer clarifying questions and pass a sanitised, enriched prompt to the existing sandboxed pipeline (Section 3.2). The second is stronger sandbox isolation through a hardened default container and an optional gVisor or microVM runtime. The third is improved monitoring with an event-driven file watcher and a cross-check of the event log against the trusted diff. The fourth is role-based approval, which separates developer and reviewer roles so that approval becomes a separation-of-duties control. These enhancements follow directly from the limitations measured in Chapters 6 and 7.

Table 4.1: Planned project schedule for VaultShield

Sr.	Project activity	Duration	Target period
Semester VII			
1	Group formation and topic finalisation; identifying the scope and objectives of the project	1 week	10 Jul – 17 Jul 2026
2	Discussing the project topic with the help of a paper prototype	1 week	17 Jul – 24 Jul 2026
3	Identifying the functionalities of the project as per the objectives identified	1 week	24 Jul – 31 Jul 2026
4	Literature review and research paper study	1 week	31 Jul – 7 Aug 2026
5	Designing the graphical user interface	3 weeks	7 Aug – 28 Aug 2026
6	Presentation I	1 day	4 Sep 2026
7	Policy logic development	1 week	18 Sep – 25 Sep 2026
8	Core model development and training	1 week	25 Sep – 1 Oct 2026
9	Presentation II	1 day	9 Oct 2026
10	Incorporating the suggestions from Presentation II and submitting a draft copy to the project guide for proofreading	1 week	16 Oct – 23 Oct 2026
11	Draft submission to the guide for review	2 days	23 Oct – 24 Oct 2026
12	Final black book submission	1 week	26 Oct – 30 Oct 2026
13	External presentation and viva voce	1 day	2 Nov 2026
Semester VIII			
14	Implementation of the Agentic AI Orchestrator (Objective 2)	3 weeks	January 2027
15	Stronger sandbox isolation, improved monitoring and fixes from testing	3 weeks	January – February 2027
16	Role-based approval; evaluation with real agents and the Orchestrator	2 weeks	February 2027
17	Usability study with reviewers and dashboard refinements	2 weeks	March 2027
18	Research paper drafting and submission	2 weeks	March 2027
19	Final report, demonstration and viva preparation	3 weeks	March – April 2027

4.3 Team Roles and Responsibilities

The team of four divided responsibilities by architectural plane, with shared ownership of testing and documentation, as recorded in Table 4.2.

Table 4.2: Team roles and responsibilities

Member	Primary role	Responsibilities
Aayush Subhash Balip	Backend and data engineering	Django REST API, MongoDB models, run state machine, artifact ingest, Celery tasks, Docker Compose stack
Krish Mahesh Patil	Frontend and user experience	React dashboard, run detail tabs, decision controls, accessibility and responsive layout
Muhammad Anis Momin	Security research and sandbox engineering	Sandbox lifecycle, credential shaping, egress proxy, trusted capture, token hygiene, threat model
Sameer Hasan Saiyed	Integration, testing and documentation	CLI pipeline, agent adapters, submit and teardown, test strategy, benchmark and report

4.4 Software Size Estimation

4.4.1 KLOC-Based Estimation

The size of the delivered software was measured directly from the repository as logical source lines, excluding blank lines, comments, docstrings, migrations, lock files and generated content. Table 4.3 reports the counts.

Table 4.3: Measured software size by component

Component	Product code, SLOC	Test code, SLOC
CLI package and tools	12,404	16,697
Backend	1,150	2,552
Frontend	4,356	2,511
Deployment configuration	160	–
Total delivered product	**18,070**	**21,760**

The delivered product therefore measures KLOC = 18.07. Test code is excluded from the primary estimate, because COCOMO sizes delivered source instructions.

4.5 Effort Estimation Using COCOMO

4.5.1 Basic COCOMO Model

The Basic Constructive Cost Model [15] estimates effort and schedule from size alone. VaultShield was classified as an organic project, built by a small team in a familiar environment with flexible requirements, so the coefficients are a = 2.4, b = 1.05, c = 2.5 and d = 0.38. E is effort in person-months, T is development time in months and P is the average number of persons.

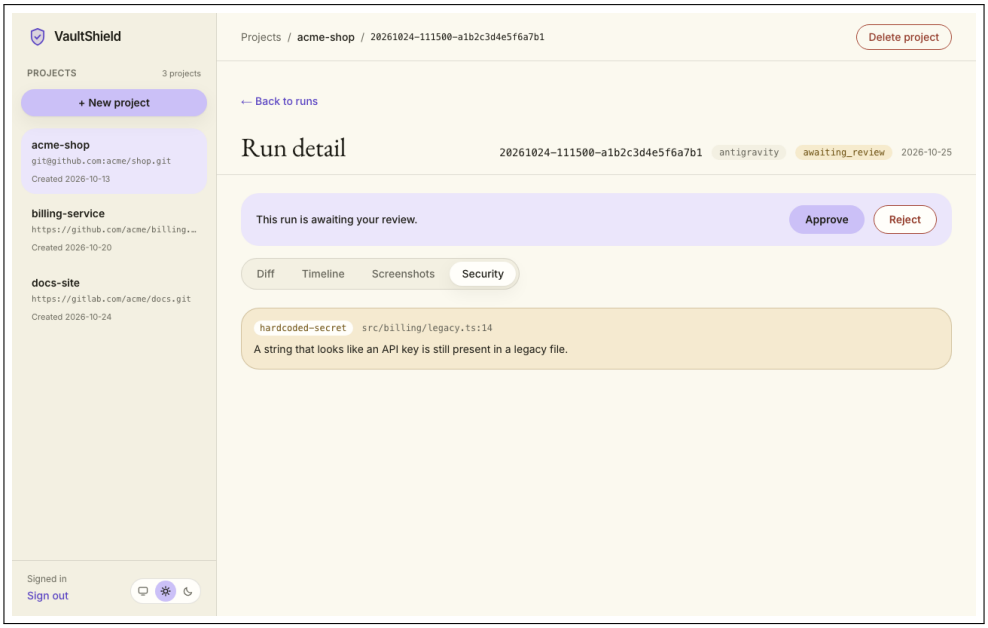


Figure 5.14: Run detail – Security tab

5.3.5 Approval Decision and Submission

When the reviewer approves a run, the buttons are replaced by the status “approved, awaiting submission” and the exact vaultshield submit command, as shown in Figure 5.15, because the backend never opens a pull request on its own.

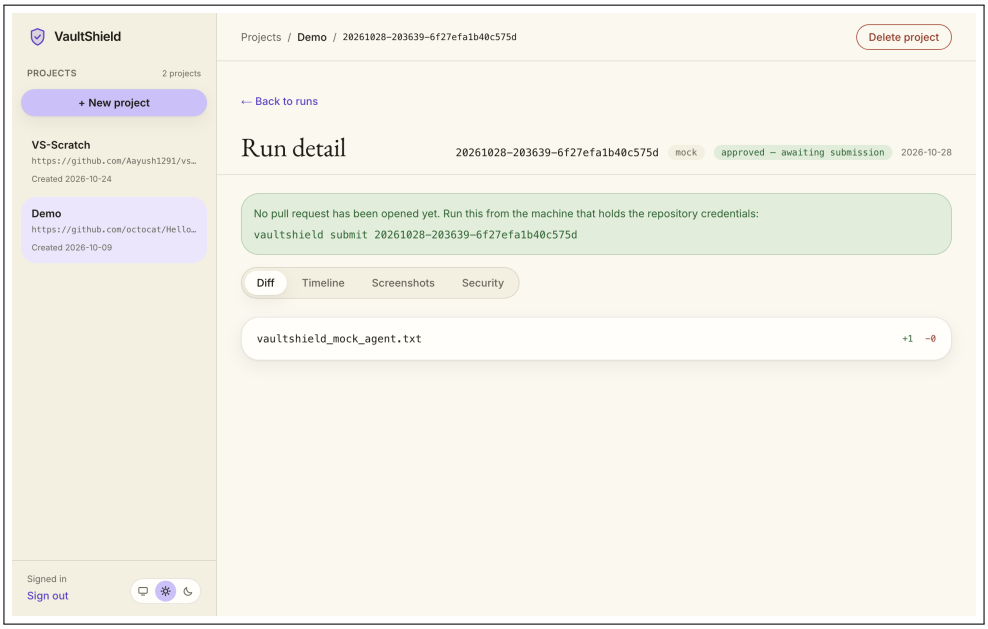


Figure 5.15: Run detail after approval, showing the submit command

5.3.6 Implementation Summary

The CLI creates the sandbox, monitors the agent, captures the change on a dedicated branch and uploads the evidence, while the dashboard presents that evidence and records the reviewer’s decision. Producing a change and accepting it therefore remain separate

steps. Table 5.1 lists every feature implemented and evaluated in Semester VII, and Table 5.2 lists the enhancements planned for Semester VIII, none of which is part of the delivered system.

Table 5.1: Implemented in Semester VII

Area	Implemented feature
CLI and adapters	Python CLI (init, login, run, submit, teardown); mock, codex, kiro and antigravity agent adapters
Sandbox execution	Docker-based sandbox per run; fresh repository clone inside the container; guaranteed teardown
Credentials	Dummy credential substitution in a read-only .env.sandbox; one-shot repository clone credentials
Egress control	Per-run internal network and hardened proxy with log-only and allowlist modes
Monitoring	Terminal (PTY) and in-container file monitoring; JSON Lines event logging; off-limits path enforcement
Change capture	Trusted change capture in a separate helper; one-commit run branch and structured Git diff
Backend and dashboard	Django REST backend with MongoDB integration; React review dashboard
Security and approval	Token-hygiene and static security scanning; human approval workflow; approval-gated pull or merge request submission
Protect module	Keychain-backed .env protection for agents run outside the sandbox

Table 5.2: Planned for Semester VIII (not implemented)

Enhancement	Planned work	Semester VII gap addressed
Agentic AI Orchestrator	Context scanner, sanitiser, clarifying questions and enriched prompt (Section 3.2)	Objective 2 not implemented
Stronger sandbox isolation	Non-root user, dropped capabilities, resource limits, default-deny egress, gVisor or microVM runtime	Shared kernel; root in the default sandbox; opt-in egress
Improved monitoring	Event-driven file watcher, event-log cross-check against the diff, live streaming, sign-in watchdog	Watcher missed 8 of 40 writes; failed test 1
Role-based approval	Reviewer accounts, role-based access control, gated status transitions	Single-user trust domain; status PATCH can set approved

5.4 Timeline Sem VII

The Gantt chart in Figure 5.16 shows the Semester VII activities across thirteen weeks, from group formation on 10 July 2026 to the external presentation and viva voce on 2 November 2026. The activities are grouped into four phases: project conception and initiation, design and implementation, evaluation and documentation, and final submission. All four members are owners of every activity, and every Semester VII activity is complete.

blocked, that a tampered in-container Git cannot hide a file from the diff, and that concurrent approve and reject requests resolve to exactly one decision.

6.2.2 System Testing

System testing drove the real vaultshield run command end to end with a real container, clone, agent process, trusted capture and teardown, and the same harness executed the 40 timed runs reported in Chapter 7.

6.3 Testing Results and Findings

Table 6.2 summarises the results of the three automated suites, which form the Semester VII testing campaign.

Table 6.2: Automated test suite results

Suite	Tests	Passed	Failed	Pass rate
CLI	2,080	2,078	2	99.90%
Backend	341	341	0	100.00%
Frontend	283	283	0	100.00%
Total	**2,704**	**2,702**	**2**	**99.93%**

No test was skipped. The lowest text contrast was 4.77:1 in the light theme and 5.80:1 in the dark theme, above the 4.5:1 threshold, and the layout report recorded no horizontal overflow across 63 page and width combinations. Table 6.3 lists the main issues found during development and testing.

Table 6.3: Issues found and their resolution

Issue	Resolution	Status
54 of 72 losing concurrent decisions received HTTP 500 instead of 409	Retry on transient transaction errors	Fixed
Dummy connection strings kept the real database host name	Host replaced unless provably local	Fixed
A missing credential file became an empty directory through Docker's bind mount	File check before any Docker call	Fixed
Host listeners reachable from a plain internal bridge	Internal bridge created without a gateway	Fixed
A hanging interactive sign-in was not bounded by its deadline (failed test 1)	Watchdog-based deadline planned	Open
A test expected the raw echoed value that redaction now masks (failed test 2)	Test expectation to be updated	Open
The file watcher missed the agent's write in some fast runs	The trusted diff remains the authoritative record	Open

Issue	Resolution	Status
The status PATCH route can set approved without an Approval record	Documented MVP limitation; gating planned	Open

6.3.1 Test Environment and Configuration

Table 6.4 records the configuration under which the suites and the benchmark were run.

Table 6.4: Test environment and configuration

Item	Configuration
Hardware	Apple MacBook Air, M4 processor
Container runtime	Docker Desktop 29.2.1; Docker-backed tests use the vaultshield sandbox image and the real daemon
CLI suite	Python, pytest with Hypothesis; command: cd cli && pytest -q
Backend suite	pytest against a MongoDB replica set, in a throw-away test_ database; command: cd backend && pytest -q
Frontend suite	Jest with React Testing Library and fast-check; command: cd frontend && npm test
Isolation of tests	Home directory redirected to an empty temporary directory; containers and networks removed in finally blocks
System-test agent	Deterministic mock agent; sample repository https://github.com/octocat/Hello-World.git

The figures in Table 6.2 are from the Semester VII testing campaign. The suites were re-run on 1 November 2026, as shown in Figure 6.1. The backend (341 of 341) and frontend (283 of 283) results were identical. The CLI suite collected 2,084 tests in the re-run, four more than the 2,080 of the original campaign because tests were added afterwards; 2,082 passed and the same two tests failed. The re-run confirms the campaign result; it does not replace it.



Figure 6.1: Re-run of the three automated suites on 1 November 2026 (excerpt of summary lines)

6.3.2 Analysis of the Two Failed Tests

Both failing tests are Docker-backed tests in tests/test_sandbox_execution.py. Both exercise the interactive sign-in transport, which is used only when the antigravity agent is run with --interactive-auth. This transport writes a pasted authorisation code to the agent's standard input through the Docker exec socket and redacts it from everything the CLI displays. Figure 6.2 shows both failures re-run individually.



Figure 6.2: The two failing tests re-run individually, with their durations (excerpt)

Failure 1: interactive sign-in timeout. The test test_a_hanging_signin_hits_the_read_timeout_instead_of_blocking_forever starts a fake sign-in program in a real container that prints the code prompt and then hangs. The test shortens the read deadline to 3 seconds and expects the CLI to give up within 60 seconds. The read deadline is set as a socket timeout on the exec stream, and on Docker Desktop 29.2.1 that timeout does not interrupt the blocking read, so the read blocked for 600.33 seconds until the fake program exited. This is a genuine availability defect in the optional interactive authentication flow: a sign-in that stops responding keeps the CLI waiting instead of failing after the deadline. It exposes no secret, because the code is still redacted and teardown still runs, which other passing tests verify. A watchdog-based deadline that kills the exec independently of the socket is planned.

Failure 2: redacted echo expectation. The test test_exec_interactive_really_delivers_stdin_to_a_container_process sends a sentinel value to a container process that prints RECEIVED:[value], and expects the raw value in the output. The actual output was RECEIVED:[***]. Redaction replaces only an exact match of the sent value, so this output proves that the process received exactly the bytes sent and that the terminal echo was then masked. The masking is intentional credential protection that keeps an authorisation code off the terminal and out of the logs. The test predates the redaction, so its expectation is outdated and should be changed to the redacted form.

Reliability impact. The two failures are about 0.07% of the 2,704 tests of the original campaign. They are of different kinds. Failure 1 is a genuine functional defect: the code does not do what it is specified to do, and a user of the optional sign-in flow can observe it. Failure 2 is an outdated test assertion: the code behaves as specified and only the test's expectation is wrong, so it is not a functional or security defect. Neither affects the default execution workflow, which does not use the interactive transport. The security-critical isolation, secret-substitution, egress, trusted-capture, token-hygiene and approval tests all passed; they drive the sandbox through the ordinary non-interactive exec path that every default run uses, so a defect in the optional sign-in path cannot hide a failure in them.

6.4 Sample Test Cases

Table 6.5 presents representative test cases, each corresponding to an automated test or to the benchmark harness.

Table 6.5: Sample test cases

ID	Description	Expected and actual result	Status
TC-01	Sandbox isolation	Probing agent cannot read the host .env, home directory or Docker socket; host checkout untouched in 40 of 40 runs	Pass
TC-02	Secret suppression	No generated dummy value contains its real value; 135 of 135 tests passed	Pass
TC-03	Network blocking	CONNECT to unlisted hosts, IP literals and the host answered 403; direct DNS lookups fail	Pass
TC-04	Off-limits enforcement	Write to a protected path records a violation event, then fails the run	Pass
TC-05	Tamper resistance	A file hidden by a tampered in-container Git still appears on the run branch	Pass
TC-06	Token hygiene	All five credential-leak vectors fail the run and quarantine the diff	Pass
TC-07	Approval gating	submit refuses every non-approved run with no push and no PR	Pass
TC-08	Concurrent decisions	Exactly one winner per trial and exactly one Approval record	Pass
TC-09	Monitoring completeness	Every agent write in the timeline; present in 32 of 40 runs, while the diff recorded 40 of 40	Partial
TC-10	Hanging sign-in is bounded	Expected to stop within 60 s; took 600.3 s	Fail

6.5 Conclusion on Testing

Of 2,704 automated tests in the Semester VII campaign, 2,702 passed, and every security-critical behaviour named in the objectives was exercised against real infrastructure. As analysed in Section 6.3.2, the two failures affect only the optional interactive sign-in transport: one is an availability defect, the other an outdated test assertion. The file watcher's missed events are a gap in supplementary evidence rather than in change capture, because the trusted diff was complete in every run, and the unrestricted status PATCH route is acceptable only within the single-developer trust model of the MVP.

Chapter 7

Result and Discussions

7.1 Introduction

This chapter evaluates VaultShield with a timing harness that executed 40 complete vaultshield run invocations against real Docker, a separate baseline of 20 runs of the same agent directly on the host, the security-focused test cases of Chapter 6, the feasibility spike on secret exposure, captures of a real execution, and a case study of preserved run directories. The measurements were taken on an Apple MacBook Air with an M4 processor running Docker Desktop 29.2.1, with a small local repository and a stubbed backend.

Scope of the performance results. Every timing in this chapter was measured with VaultShield's deterministic *mock agent*, not with a real model-backed agent such as Codex, Kiro or Antigravity. The mock agent runs inside the sandbox through the same adapter, PTY and monitoring path as a real agent, but it makes no model API calls and writes one small file. The results therefore measure the fixed overhead of the VaultShield pipeline under the mock-agent workload. They must not be read as the performance of real agents, whose evaluation is future work planned for Semester VIII.

7.1.1 Benchmark Configuration

Table 7.1 lists the four configurations. The benchmark consists of 60 timed executions: 40 VaultShield runs (20 default, 10 with the egress allowlist and 10 with one screenshot route) and 20 direct-host baseline runs, which did not use VaultShield and serve only to compute overhead. Every reference to "the 40 runs" in this report means the 40 VaultShield runs. The raw per-run timing records of this campaign are not preserved in the repository; the counts and figures reported here are those recorded in the benchmark campaign and documented in the project's earlier full report draft, and they have not been independently re-measured.

Table 7.1: Benchmark configurations (all with the mock agent)

Configuration	Runs	Execution	Purpose
Direct-host baseline	20	Host clone, no container	Baseline without VaultShield
VaultShield default	20	Sandbox and trusted capture, egress off	Fixed pipeline overhead
VaultShield with egress allowlisting	10	As default, egress allowlist on	Cost of network isolation
VaultShield with screenshot capture	10	As default, one screenshot route	Cost of browser capture
Total	**60**	**40 VaultShield + 20 baseline**	

7.2 Evaluation Metrics

Table 7.2 defines the metrics used.

Table 7.2: Evaluation metrics

Metric	Definition
End-to-end latency	Wall-clock time of one vaultshield run, from command start to exit
Stage latency	Time spent in each pipeline stage
Overhead	Difference between VaultShield latency and the same agent run directly on the host
Isolation effectiveness	Fraction of runs in which the host checkout was unchanged and host files were unreachable
Capture and monitoring completeness	Fraction of runs whose diff and event log contained the agent's change
Detection of unsafe actions	Fraction of injected unsafe behaviours that were blocked or detected
Cleanup reliability	Number of VaultShield containers or networks left after all runs

7.3 Performance Analysis

Table 7.3 reports the stage latency of the default configuration over 20 runs, in seconds, and Figure 7.1 shows the mean of each stage.

Table 7.3: Stage latency in the default configuration

Stage	Mean	Std. dev.	Share
Sandbox creation, clone and install	0.239	0.017	17.0%
Agent execution and monitoring	0.413	0.033	29.4%
Trusted capture	0.651	0.053	46.4%
Token-hygiene scan	0.019	0.008	1.4%
Teardown	0.042	0.006	3.0%
Other stages	0.039	–	2.8%
End-to-end total	**1.403**	**0.075**	**100%**

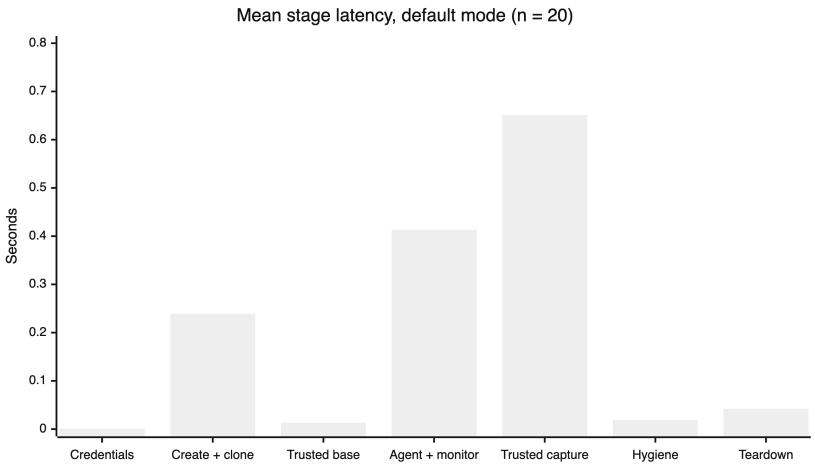


Figure 7.1: Mean stage latency in the default configuration

Trusted capture is the most expensive stage because it starts a second container, extracts the work tree and rebuilds the repository; this is the price of a diff the agent cannot falsify. With a real agent, model response time would dominate the agent-execution stage and capture costs would grow with the size of the change, so these stage shares apply only to the mock agent's one-file change. Agent execution took 0.413 s against 0.186 s on the host, the difference covering the in-container watcher and Docker's exec API. Table 7.4 and Figure 7.2 compare the configurations.

Table 7.4: End-to-end latency by configuration

Configuration	Runs	Mean time, s
Direct on host: clone and agent	20	0.237
VaultShield default	20	1.403
VaultShield with egress allowlist	10	2.213
VaultShield with one screenshot route	10	3.294

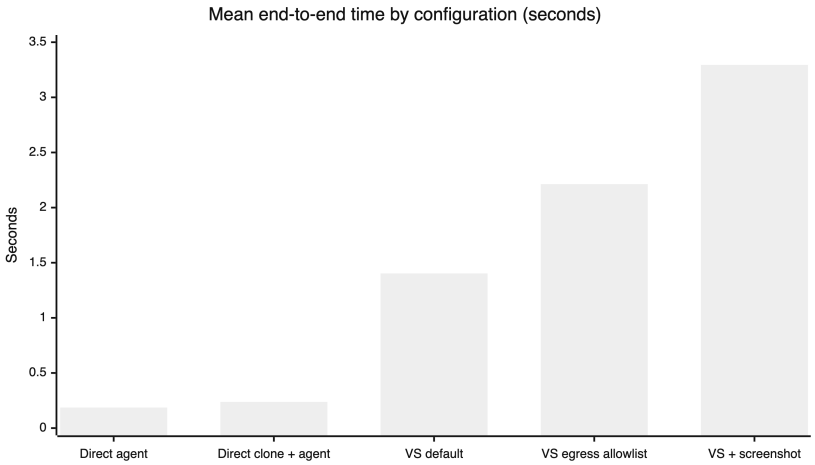


Figure 7.2: Mean end-to-end time by configuration

The egress allowlist added 0.810 s, mostly for creating the per-run networks and proxy

and removing them at teardown. One screenshot added 1.891 s, almost all of it spent starting the application and waiting for it to answer.

7.4 Overhead with and without VaultShield

Against direct execution with a fresh clone, VaultShield's default mode added 1.166 s per run and the egress allowlist mode added 1.976 s with the mock agent. If this overhead stayed fixed, it would be 0.39% and 0.66% of a five-minute agent session. This is a projection, not a measurement: it assumes that a real agent's session is unaffected by the sandbox and that capture costs do not grow with the change, neither of which has been measured. Table 7.5 places the overhead alongside the qualitative differences.

Table 7.5: Running an agent with and without VaultShield

Dimension	Direct execution	VaultShield
Fixed overhead	–	+1.166 s, or +1.976 s with egress
Host checkout unchanged	No	Yes, in every run
Real .env values visible to the agent	Yes	No
Change captured on a reviewable branch	No	Yes, in every run
Outbound traffic restricted and logged	No	With egress allowlist
Approval required before a PR	No	Yes
Containers or networks left behind	–	0

7.5 Safety Events Captured

Table 7.6 summarises the safety-relevant results across the 40 VaultShield benchmark runs and the security-focused test cases. These are rates over injected scenarios, which show that each control works for the behaviour it was designed against but do not prove the controls are exhaustive.

Table 7.6: Safety events and evidence captured

Scenario	Result
Host checkout unchanged after the run	40 of 40
Agent's file present in diff.json	40 of 40
File-change event recorded for the agent's file	32 of 40
Non-allowlisted connections blocked and logged	4 of 4
Credential-leak vectors detected	5 of 5
Tampered in-container Git defeated	2 of 2

Scenario	Result
Values in six real .env.sandbox files that are dummies	30 of 30
Containers and networks left after the campaign	0

7.6 Secret Exposure With and Without Protection

The feasibility spike asked a production coding agent to read .env and .env.local and report the exact values of two fake keys. Table 7.7 reports the outcome under four protection configurations.

Table 7.7: Secret exposure under four protection configurations

Config	Protection layers	Verdict
C0	None	Both secrets printed
C1	Agent deny rules for .env files	.env.local read into context
C2	C1 plus a hook denying shell access to .env	.env.local read into context
C3	C2 plus real values replaced by a placeholder on disk	Safe

Agent-level deny rules proved brittle, and only keeping the real value off disk was safe. This is why the sandbox receives dummy values and why the Protect module moves values into the macOS Keychain for agents run outside the sandbox. Unlike the benchmark, this experiment used a real production coding agent, but a single agent and a single prompt, so it shows that deny rules can fail rather than how often they fail.

7.7 Case Study: Analysis of Real Runs

The repository preserves five complete mock-agent runs from development. In run 20261009-091725-ff669fcd7dd00b2b, the event log records seven events: the agent confirms that its working directory is the sandbox clone, narrates its work over 157.9 ms, and the in-container watcher then independently reports the created file 106.5 ms after the agent's write. The diff contains exactly that file, and the screenshot shows the sandbox clone's directory listing. In one of the five runs, however, the event log has no file-change event even though the diff and screenshot both show the file, the same gap seen in 8 of the 40 benchmark runs. The event log is therefore corroborating evidence of behaviour, while the trusted diff is the authoritative record of what changed.

7.7.1 Execution Evidence from a Live Run

To complement the benchmark tables, the complete system was executed on 1 November 2026 against the running Docker Compose stack and the public repository octocat/Hello-World, using the mock agent with the egress allowlist enabled. The figures below are

captures of those real executions. Terminal output was saved from the commands and rendered in a terminal frame without editing; where an output was long, the figure shows an excerpt of selected lines, marked as such in the figure. The dashboard images are screenshots of the live dashboard, cropped to the run panel.

Figure 7.3 shows the run itself. The run was registered with the backend before the sandbox was created, the dummy credential file was built from the developer's .env, the sandbox was moved onto the egress-isolated network, the change was committed to the run branch in the capture helper, and the sandbox was torn down before the evidence was uploaded. The whole command took 6.89 s of wall-clock time including the clone from GitHub, the backend upload and one screenshot.

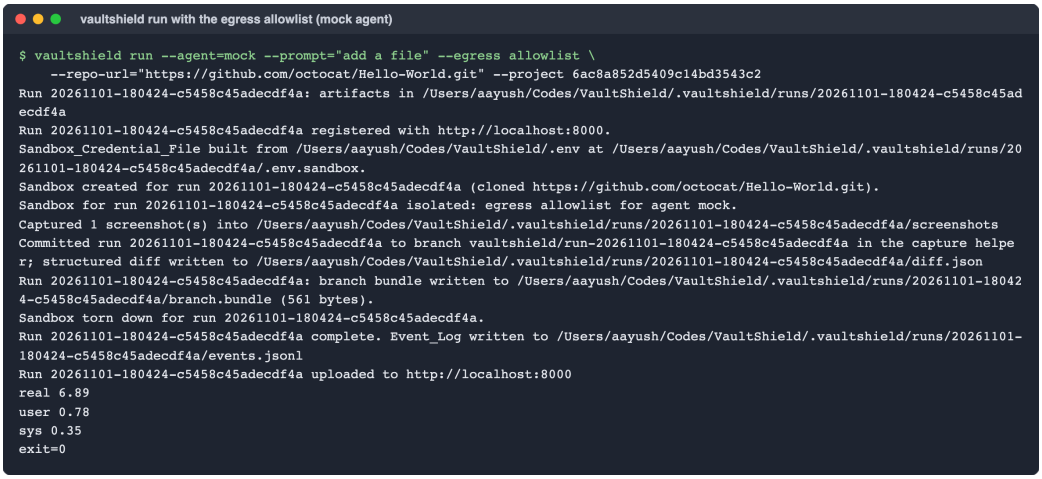


Figure 7.3: A real vaultshield run with the egress allowlist and the mock agent

Figure 7.4 shows the artifacts that run produced. Every value in .env.sandbox is a dummy of the same shape as the real value, including a database URL whose credentials and host were replaced, and the event log records the agent's output lines and the watcher's independent file-change event. The structured diff contains exactly the one file the agent created.

Figure 7.5 and Figure 7.6 show the same run in the live dashboard, awaiting review, with its timeline and diff.

Figure 7.7 shows the approval gate and the cleanup check. Submitting the run while it was still awaiting review was refused before any side effect, no VaultShield sandbox, capture helper or proxy container remained, the only remaining network was the Compose stack's own, and the agent's file did not exist in the host checkout.

Figure 7.8 shows the isolation probe, a standalone script that creates a real sandbox through the same class the run command uses and tries what a hostile agent might try. The host .env and the Docker socket were absent, the working directory was the sandbox clone, and only the dummy credential file was readable. The home directory listed by the probe is the container's own /root, which contains only image defaults.

Figure 7.9 shows the Docker-backed security tests run against real containers: a blocked connection is blocked and recorded, DNS exfiltration is blocked, all five token-leak vectors are detected, and a tampered in-container Git cannot hide a file from the branch or diff. All 15 tests passed in 34.4 s.

Artifacts written by the run

```
$ cat .env.sandbox            # dummy values only
VAULTSHIELD_MONGODB_URL=mongodb+srv://dummy_user:dummy_password@localhost/?retryWrites=true&w=majority
VAULTSHIELD_MONGODB_NAME=dummy-value
SECRET_KEY=dummy-value
DEBUG=dummy-value
ALLOWED_HOSTS=dummy-value
GEMINI_API_KEY=dummy-value
GROQ_API_KEY=z6uyz8QLg1ajKg9BC9ZKJAqDIOmvIZOwicsbOb1CGETWdwEKRk8OFdzD

$ cut -c1-150 events.jsonl     # lines cut at 150 characters
{"event_type": "output", "timestamp": "2026-11-01T18:04:26.582646+00:00", "payload": "[mock-sandbox] cwd=/workspace/repo"}
{"event_type": "output", "timestamp": "2026-11-01T18:04:26.582706+00:00", "payload": "[mock-sandbox step=0] simulated output f
or prompt='add a file'"}
{"event_type": "output", "timestamp": "2026-11-01T18:04:26.637693+00:00", "payload": "[mock-sandbox step=1] simulated output f
or prompt='add a file'"}
{"event_type": "output", "timestamp": "2026-11-01T18:04:26.693660+00:00", "payload": "[mock-sandbox step=2] simulated output f
or prompt='add a file'"}
{"event_type": "output", "timestamp": "2026-11-01T18:04:26.750056+00:00", "payload": "[mock-sandbox] wrote vaultshield_mock_ag
ent.txt"}
{"event_type": "output", "timestamp": "2026-11-01T18:04:26.750116+00:00", "payload": "[mock-sandbox] simulated final message f
or prompt='add a file'"}
{"event_type": "file_change", "timestamp": "2026-11-01T18:04:26.862517Z", "payload": "created: /workspace/repo/vaultshield_moc
k_agent.txt"}

# diff.json, pretty-printed with Python's json module, lines cut at 150 characters
{
 "files": [
  {
   "path": "vaultshield_mock_agent.txt",
   "additions": 1,
   "deletions": 0,
   "patch": "diff --git a/vaultshield_mock_agent.txt b/vaultshield_mock_agent.txt\nnew file mode 100644\nindex 0000000..f59c89
5\n--- /dev/null\n+++ b/
  }
 ]
}
```

Figure 7.4: Run artifacts: dummy credential file, event log and structured diff

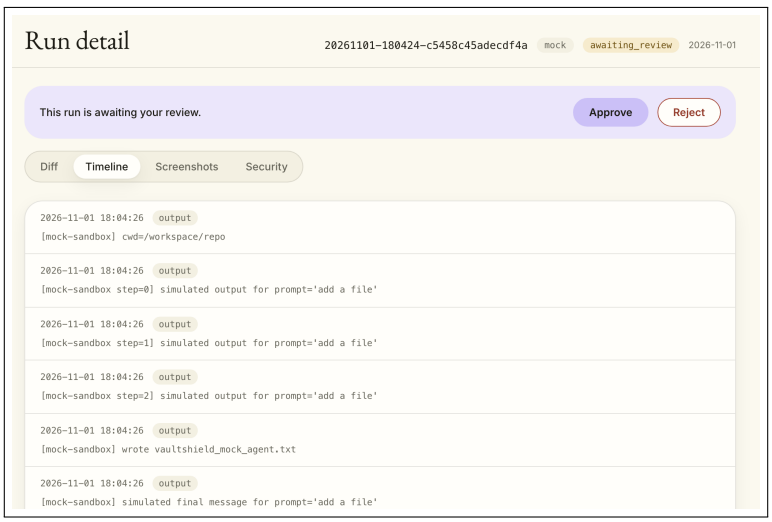


Figure 7.5: Live dashboard: Timeline tab of the real run



Figure 7.6: Live dashboard: Diff tab of the real run, awaiting review

Approval gate and cleanup check after the run

```
$ vaultshield submit 20261101-180424-c5458c45adecdf4a
Error: Run 20261101-180424-c5458c45adecdf4a is 'awaiting_review', not 'approved', so it cannot be submitted. Nothing was pushed
and no pull request was opened.
exit=1

$ docker ps -a --filter name=vaultshield-run --filter name=vaultshield-capture --filter name=vaultshield-egress
NAMES     STATUS

$ docker network ls --filter name=vaultshield
NAME                 DRIVER
vaultshield_default   bridge

$ ls ../vaultshield_mock_agent.txt   # run from cli/: the host checkout root
ls: ../vaultshield_mock_agent.txt: No such file or directory
```

Figure 7.7: Approval gate refusing an unapproved run, and no leftover containers or host changes

isolation_probe.py: hostile-agent probes in a real sandbox

```
$ python isolation_probe.py   # excerpt: probe headings and errors
[+] Creating Sandbox (base_image=python:3.11, repo=https://github.com/octocat/Hello-World.git) ...
[+] Sandbox created. Running probes:
--- Current working directory [OK] ---
/workspace/repo
--- List cwd contents [OK] ---
--- Read host .env (should FAIL) [exit=1] ---
cat: /host/.env: No such file or directory
--- Read mounted .env.sandbox (should WORK) [OK] ---
--- List host home directory (should FAIL) [OK] ---
--- List /home (should be empty/absent) [OK] ---
--- Access Docker socket (should FAIL) [exit=2] ---
ls: cannot access '/var/run/docker.sock': No such file or directory
--- Attempt Docker socket read via curl (should FAIL) [OK] ---
--- Network identity / hostname [OK] ---
--- Write a test file into the repo [OK] ---
[+] Tearing down Sandbox ...
[+] Done.
```

Figure 7.8: Isolation probe run inside a real sandbox (excerpt of probe headings and errors)

Docker-backed egress, token-hygiene and tamper tests

```
$ pytest -v tests/test_egress_docker.py   # excerpt: result lines
tests/test_egress_docker.py::test_a_blocked_connection_is_blocked_and_recorded PASSED [  6%]
tests/test_egress_docker.py::test_an_allowlisted_host_is_reachable PASSED [ 13%]
tests/test_egress_docker.py::test_dns_exfil_is_blocked PASSED            [ 20%]
tests/test_egress_docker.py::test_the_app_port_is_reachable_through_the_proxy PASSED [ 26%]
tests/test_egress_docker.py::test_run_with_egress_allowlist_end_to_end PASSED [ 33%]
tests/test_egress_docker.py::test_token_hygiene_vectors_in_a_real_container[copy] PASSED [ 40%]
tests/test_egress_docker.py::test_token_hygiene_vectors_in_a_real_container[core_worktree] PASSED [ 46%]
tests/test_egress_docker.py::test_token_hygiene_vectors_in_a_real_container[symlink] PASSED [ 53%]
tests/test_egress_docker.py::test_token_hygiene_vectors_in_a_real_container[gitignored] PASSED [ 60%]
tests/test_egress_docker.py::test_token_hygiene_vectors_in_a_real_container[tracked_symlink] PASSED [ 66%]
tests/test_egress_docker.py::test_credential_snapshot_never_follows_symlinks_out_of_the_root PASSED [ 73%]
tests/test_egress_docker.py::test_a_tampered_in_container_git_cannot_hide_a_file_from_the_branch_or_diff[True] PASSED [ 80%]
tests/test_egress_docker.py::test_a_tampered_in_container_git_cannot_hide_a_file_from_the_branch_or_diff[False] PASSED [ 86%]
tests/test_egress_docker.py::test_trusted_capture_preserves_non_utf8_and_case_variant_names[A] PASSED [ 93%]
tests/test_egress_docker.py::test_trusted_capture_preserves_non_utf8_and_case_variant_names[B] PASSED [100%]
========================== 15 passed in 34.40s ==========================
```

Figure 7.9: Docker-backed egress, token-hygiene and tamper-resistance tests (excerpt of result lines)

7.8 Overall Discussion and Limitations

With the mock agent, separating the execution of an agent from trust in its output cost about 1.2 seconds per run, or about 2.0 seconds with network allowlisting. In exchange, every evaluated run left the developer's checkout untouched, kept real secrets out of the agent's environment, recorded an event log, captured the change on a branch the agent could not falsify and held it behind an approval state machine. Table 7.8 lists the main residual risks.

Table 7.8: Residual risks and limitations

Limitation	Consequence	Planned improvement
Shared-kernel container isolation; default sandbox runs as root	An exploit could reach the host	Default hardening, gVisor or microVMs
Egress unrestricted unless enabled	Repository content could be exfiltrated	Default-deny egress
Host-based allowlist	Uploads to an allowlisted service are possible	Path-aware filtering
Status PATCH can set approved; no multi-user model	A token holder can skip review	Gated transitions, reviewer roles
Polling file watcher misses some writes	Timeline may omit a change	Diff cross-check, event-driven watcher
Diff tab shows counts, not patch text	Line-level review needs diff.json	Inline patch rendering

The evaluation itself is limited. All timing figures come from the mock agent on one machine and one small repository, so they describe the pipeline's fixed overhead and must not be generalised to Codex, Kiro, Antigravity or any other model-backed agent. The safety results are rates over injected scenarios and do not measure resistance to an adaptive adversary. Evaluation with real agents is planned for Semester VIII.

7.8.1 Comparison with Existing Approaches

Table 7.9 compares the outcome of the same threats under three approaches: running the agent directly on the host, relying on agent-native protection (deny rules, OS sandbox and approval prompts, as documented for Claude Code and Codex CLI [16], [18]), and VaultShield. The direct-execution and VaultShield columns, and the deny-rule result, were measured in this project. The remaining agent-native entries come from vendor documentation and were not measured by us, and "not addressed" means the cited documentation does not describe the control. The product-level comparison of Table 2.2 is not repeated here.

Table 7.9: Outcome of the same threats under three approaches

Threat or property	Direct on host (measured)	Agent-native protection	VaultShield (measured)
Agent reads a secret in the project .env	Both secrets printed (C0)	Deny rules bypassed via .env.local (C1, C2, measured)	No real value present; 30 of 30 values dummies

Threat or property	Direct on host (measured)	Agent-native protection	VaultShield (measured)
Agent modifies the developer's checkout	Yes	Yes, edits land in the workspace by design	Host unchanged in 40 of 40 runs
Outbound connection to an unlisted host	Allowed	Blocked by default in Codex CLI; configurable in Claude Code	Blocked and logged, 4 of 4, when the allowlist is on
Agent hides a change by tampering with Git	Possible	Not addressed in cited documentation	Defeated, 2 of 2
Agent's credential leaks into its output	Not checked	Not addressed in cited documentation	Detected, 5 of 5 vectors
Change reaches the repository without review	Yes, if the developer commits	Per-action prompts; no held change	Refused until approved (Fig. 7.7)
Works with several agents	–	No, built into one agent	Yes, four adapters
Fixed time cost per run (mock agent)	0.237 s	Not measured	+1.166 s, or +1.976 s with egress

The comparison shows where VaultShield adds protection that the other approaches do not provide, and also that agent-native sandboxes already block the network by default where VaultShield requires opting in. The two kinds of control are complementary: an agent's own sandbox can still run inside a VaultShield sandbox.

7.9 Chapter Summary

With the mock agent, the default pipeline completed in 1.403 s on average, adding 1.166 s to direct execution, with trusted capture the largest stage; real-agent timings remain to be measured. Isolation, change capture and cleanup succeeded in every evaluated run, all injected unsafe behaviours were blocked or detected, and every value in six real sandbox credential files was a dummy. The file watcher recorded the agent's change in 80% of runs, confirming the trusted diff as the authoritative record.

Chapter 8

Conclusion

VaultShield separates running an AI coding agent from trusting what it produces. The agent receives a disposable Docker container, a fresh clone and only dummy credentials; its output, file changes and network decisions are recorded in an event log; its change is captured as one commit on a dedicated branch by a separate hardened container; and the change can become a pull or merge request only after a human approves it in the review dashboard. The system was built as a Python CLI, a Django REST Framework backend on MongoDB with Celery workers, and a React dashboard, totalling about 18.1 thousand logical lines of product code. Table 8.1 assesses the outcome against each objective.

Table 8.1: Achievement of objectives

Objective	Evidence	Outcome
1. Universal agent execution platform	Mock, codex, kiro and antigravity agents run behind one adapter interface	Achieved
2. Agentic AI Orchestrator	Not implemented in Semester VII; proposed design in Section 3.2	Planned for Semester VIII
3. Secure sandboxed execution	Host checkout and .env unchanged in 40 of 40 runs; dummy credentials and one-shot clone token	Achieved
4. Monitoring and security analysis	Event log and diff for every run; violations recorded; Celery security scan	Achieved, with a file-watcher gap
5. Governance and human approval	Dashboard review and decision controls; submit refuses non-approved runs	Achieved

Objectives 1, 3, 4 and 5 were achieved in Semester VII to the extent supported by the evaluation, which used a mock agent for all timings. Objective 2, the Agentic AI Orchestrator, was not implemented in Semester VII and remains planned for Semester VIII; its proposed design, workflow and security considerations are given in Chapter 3 (Section 3.2 and Figure 3.2). Known limitations remain: the shared-kernel container boundary and opt-in egress control (Section 3.3), and two further gaps: in the single-developer MVP a token holder can bypass the dashboard through an unrestricted status update, and the polling file watcher missed some writes in very short runs, although the trusted diff captured every change. The project contributes a working implementation of the governance practices recommended for agentic systems [9], [10], engineering techniques such as shape-preserving credential substitution, a one-shot clone credential and trusted capture, and evidence that the pipeline's own fixed cost is about 1.2 seconds per run with a mock agent, while agent-level deny rules alone do not protect secrets. Real-agent performance has not yet been measured.

Chapter 9

Future Scope

The limitations identified in this report define the future work. The first four items are the committed Semester VIII enhancements, each tied to a limitation found in the evaluation; the remaining items are longer-term work.

Semester VIII enhancements

1. **Agentic AI Orchestrator:** implement the design in Section 3.2, which scans the project, asks clarifying questions before execution and passes a sanitised, enriched prompt to the agent, then measure on real agents whether it raises task success and approval rates and reduces hallucinated files, functions and packages. This completes Objective 2.
2. **Stronger Sandbox Isolation:** run the sandbox by default as a non-root user with all capabilities dropped, no-new-privileges and CPU, memory and process limits; make egress default-deny; and support gVisor or Firecracker microVM runtimes [12], [13]. This addresses the shared-kernel and opt-in egress limitations (threats T3, T7 and T8 in Section 3.3).
3. **Improved Monitoring:** replace the polling file watcher, which missed 8 of 40 short writes, with an event-driven watcher; cross-check the event log against the trusted diff; stream events live so a reviewer can interrupt a run [9]; render patch text in the Diff tab; and enforce the interactive sign-in deadline with a watchdog (Section 6.3.2).
4. **Role-Based Approval:** add separate developer and reviewer accounts, role-based access control, decision attribution and optional multiple approvals, and route every status transition through the gated approval endpoint, so that the approval gate becomes a separation-of-duties control rather than a single-user workflow control (threat T9).

Further long-term work

- **Policy Engine:** express rules such as required approvals for infrastructure files or approved dependency lists declaratively, with path-aware egress filtering.
- **Cost and Resource Controls:** record per-run token usage and enforce per-project budgets and sandbox time limits.
- **Multi-Repository and Parallel Runs:** capture one branch per repository in a single run, and let several agents attempt the same task for the reviewer to compare.
- **IDE Integration and Evidence Quality:** start and review runs from the editor, and scan screenshots for credentials.
- **Broader Evaluation:** extend the real-agent evaluation to a subset of SWE-bench [2] and to adaptive red-team attacks, with a usability study with reviewers.

Bibliography

[1] Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., et al., "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, 2021.

[2] Jimenez, C. E., Yang, J., Wettig, A., Yao, S., Pei, K., Press, O., and Narasimhan, K., "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?," International Conference on Learning Representations, 2024.

[3] Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O., "SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering," Advances in Neural Information Processing Systems, 2024.

[4] Wang, X., Li, B., Song, Y., Xu, F. F., Tang, X., Zhuge, M., et al., "OpenHands: An Open Platform for AI Software Developers as Generalist Agents," International Conference on Learning Representations, 2025.

[5] Ruan, Y., Dong, H., Wang, A., Pitis, S., Zhou, Y., Ba, J., Dubois, Y., Maddison, C. J., and Hashimoto, T., "Identifying the Risks of LM Agents with an LM-Emulated Sandbox," International Conference on Learning Representations, 2024.

[6] Debenedetti, E., Zhang, J., Balunović, M., Beurer-Kellner, L., Fischer, M., and Tramèr, F., "AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents," Advances in Neural Information Processing Systems, Datasets and Benchmarks Track, 2024.

[7] Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., and Fritz, M., "Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection," ACM Workshop on Artificial Intelligence and Security, 2023.

[8] Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., and Karri, R., "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," IEEE Symposium on Security and Privacy, 2022.

[9] Shavit, Y., Agarwal, S., Brundage, M., et al., "Practices for Governing Agentic AI Systems," OpenAI White Paper, 2023.

[10] Chan, A., Ezell, C., Kaufmann, M., Wei, K., Hammond, L., Bradley, H., et al., "Visibility into AI Agents," ACM Conference on Fairness, Accountability, and Transparency, 2024.

[11] Combe, T., Martin, A., and Di Pietro, R., "To Docker or Not to Docker: A Security Perspective," IEEE Cloud Computing, 2016.

[12] Agache, A., Brooker, M., Florescu, A., Iordache, A., Liguori, A., Neugebauer, R., Piwonka, P., and Popa, D.-M., "Firecracker: Lightweight Virtualization for Serverless Applications," USENIX Symposium on Networked Systems Design and Implementation, 2020.

[13] Young, E. G., Zhu, P., Caraza-Harter, T., Arpaci-Dusseau, A. C., and Arpaci-Dusseau, R. H., "The True Cost of Containing: A gVisor Case Study," USENIX Workshop on Hot Topics in Cloud Computing, 2019.

[14] Bacchelli, A., and Bird, C., "Expectations, Outcomes, and Challenges of Modern Code Review," International Conference on Software Engineering, 2013.

[15] Boehm, B. W., "Software Engineering Economics," Prentice-Hall, 1981.

[16] Anthropic, "Configure the Sandboxed Bash Tool," Claude Code documentation, accessed 1 November 2026.

[17] Anthropic, "Making Claude Code Safer with Sandboxing," Anthropic, accessed 1 November 2026.

[18] OpenAI, "Agent Approvals and Security – Codex," OpenAI developer documentation, accessed 1 November 2026.

[19] GitHub, "Responsible Use of GitHub Copilot Cloud Agent on GitHub.com," GitHub Docs, accessed 1 November 2026.

[20] Docker, "Get Started with Local Docker Sandboxes," Docker documentation, accessed 1 November 2026.

[21] OpenHands, "Runtime Architecture," OpenHands documentation, accessed 1 November 2026.